Solution 3: Manual Dependency Purge

this solution worked for me tell me the procedure with this solution in it

Perfect! I'm glad Solution 3 worked for you. Here's the **complete updated procedure** with the dependency purge step included:

Complete Step-by-Step Procedure: CloudSim Plus Setup on Your PC

Step 1: Install Java JDK 17

Download Java JDK

1. Go to Oracle's official website: https://www.oracle.com/java/technologies/downloads/[1]
2. Scroll down to find **Java 17** (LTS version)
3. Select your operating system (Windows/macOS/Linux)
4. Download the installer (for Windows, choose the **x64 Installer**)

Install Java JDK

1. Run the downloaded installer
2. Click **Next** through the installation wizard
3. Note the installation path (default: `C:\Program Files\Java\jdk-17`)
4. Click **Next** and wait for installation to complete

Set Environment Variables (Windows)

1. Right-click **This PC** → Select **Properties**
2. Click **Advanced system settings** → Click **Environment Variables**
3. Under **System Variables**, click **New** and create:
   - Variable name: `JAVA_HOME`
   - Variable value: `C:\Program Files\Java\jdk-17` (your JDK path)
4. Find the **Path** variable, click **Edit**, then **New**, and add: `%JAVA_HOME%\bin`
5. Click **OK** on all windows

Verify Installation

1. Open **Command Prompt** (search for "cmd")
2. Type: `java -version`
3. You should see Java version 17.x.x displayed

Step 2: Install Eclipse IDE

Download Eclipse

1. Go to: https://www.eclipse.org/downloads/[2]
2. Click **Download** button to get the Eclipse Installer
3. Save the installer to your computer

Install Eclipse

1. Run the **Eclipse Installer**
2. Select **Eclipse IDE for Java Developers**
3. Choose the JDK 17 you installed (it should auto-detect)
4. Select installation folder (default is fine)
5. Click **Install** and accept the license agreement
6. Wait for installation to complete (5-10 minutes)
7. Click **Launch** when done

Step 3: Create Maven Project in Eclipse

Create New Maven Project

1. Open Eclipse IDE
2. Go to **File** → **New** → **Maven Project**
3. Check **Create a simple project (skip archetype selection)**
4. Click **Next**
5. Fill in the project details:
   - **Group Id**: `com.cloudsim`
   - **Artifact Id**: `cloudsim-simulation`
   - **Version**: `1.0.0` (default)
   - **Packaging**: `jar`
6. Click **Finish**

Step 4: Add CloudSim Plus Dependency

Edit pom.xml

1. In Eclipse, locate **pom.xml** in your project root
2. Double-click to open it
3. Click on the **pom.xml** tab (bottom of editor) to view XML source
4. Replace the entire content with this:[3] [4]

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.cloudsim</groupId>
    <artifactId>cloudsim-simulation</artifactId>
    <version>1.0.0</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.cloudsimplus</groupId>
            <artifactId>cloudsimplus</artifactId>
            <version>8.5.7</version>
        </dependency>
    </dependencies>
</project>
```

5. Save the file (**Ctrl + S**)

Step 5: Download Dependencies (IMPORTANT - Manual Purge Method)

Force Download with Dependency Purge

1. **Right-click on your project** (`cloudsim-simulation`)
2. Select **Run As** → **Maven build...**
3. In the **Goals** field, type exactly: `dependency:purge-local-repository clean install`[5]
4. Click **Run**
5. Wait for Maven to download all dependencies (check the **Console** tab)
6. You should see **BUILD SUCCESS** at the end[6]

Verify Dependencies Downloaded

1. In **Package Explorer**, expand your project
2. Look for **Maven Dependencies** folder
3. Expand it - you should see `cloudsimplus-8.5.7.jar` and several other JAR files[7] [8]

Step 6: Create Simple Simulation Code

Create Package

1. Right-click on **src/main/java** folder
2. Select **New** → **Package**
3. Enter package name: `com.cloudsim.examples`
4. Click **Finish**

Create Java Class

1. Right-click on the **com.cloudsim.examples** package
2. Select **New** → **Class**
3. Enter:
   - **Name**: `BasicSimulation`
   - Check **public static void main(String[] args)**
4. Click **Finish**

Add Simulation Code

Copy and paste this complete working example:[9] [10]

```
package com.cloudsim.examples;

import org.cloudsimplus.brokers.DatacenterBroker;
import org.cloudsimplus.brokers.DatacenterBrokerSimple;
import org.cloudsimplus.builders.tables.CloudletsTableBuilder;
import org.cloudsimplus.cloudlets.Cloudlet;
import org.cloudsimplus.cloudlets.CloudletSimple;
import org.cloudsimplus.core.CloudSimPlus;
import org.cloudsimplus.datacenters.Datacenter;
import org.cloudsimplus.datacenters.DatacenterSimple;
import org.cloudsimplus.hosts.Host;
import org.cloudsimplus.hosts.HostSimple;
import org.cloudsimplus.resources.Pe;
import org.cloudsimplus.resources.PeSimple;
import org.cloudsimplus.utilizationmodels.UtilizationModelDynamic;
import org.cloudsimplus.vms.Vm;
import org.cloudsimplus.vms.VmSimple;
```

```
import java.util.ArrayList;
import java.util.List;

public class BasicSimulation {
    private static final int HOSTS = 2;
    private static final int HOST_PES = 8;
    private static final int VMS = 2;
    private static final int CLOUDLETS = 4;

    private CloudSimPlus simulation;
    private DatacenterBroker broker;
    private List<Vm> vmList;
    private List<Cloudlet> cloudletList;

    public static void main(String[] args) {
        new BasicSimulation();
    }

    private BasicSimulation() {
        System.out.println("Starting CloudSim Plus Simulation...");

        simulation = new CloudSimPlus();

        Datacenter datacenter = createDatacenter();
        broker = new DatacenterBrokerSimple(simulation);

        vmList = createVms();
        cloudletList = createCloudlets();

        broker.submitVmList(vmList);
        broker.submitCloudletList(cloudletList);

        simulation.start();

        List<Cloudlet> finishedCloudlets = broker.getCloudletFinishedList();
        new CloudletsTableBuilder(finishedCloudlets).build();

        System.out.println("Simulation finished!");
    }

    private Datacenter createDatacenter() {
        List<Host> hostList = new ArrayList<>(HOSTS);
        for(int i = 0; i < HOSTS; i++) {
            Host host = createHost();
            hostList.add(host);
        }

        return new DatacenterSimple(simulation, hostList);
    }

    private Host createHost() {
        List<Pe> peList = new ArrayList<>(HOST_PES);
        for(int i = 0; i < HOST_PES; i++) {
            peList.add(new PeSimple(1000));
        }
```

```
        long ram = 2048; // MB
        long bw = 10000; // Mbps
        long storage = 1000000; // MB

        return new HostSimple(ram, bw, storage, peList);
    }

    private List<Vm> createVms() {
        List<Vm> list = new ArrayList<>(VMS);
        for(int i = 0; i < VMS; i++) {
            Vm vm = new VmSimple(1000, 2);
            vm.setRam(512).setBw(1000).setSize(10000);
            list.add(vm);
        }
        return list;
    }

    private List<Cloudlet> createCloudlets() {
        List<Cloudlet> list = new ArrayList<>(CLOUDLETS);
        UtilizationModelDynamic utilizationModel = new UtilizationModelDynamic(0.5);

        for(int i = 0; i < CLOUDLETS; i++) {
            Cloudlet cloudlet = new CloudletSimple(10000, 2);
            cloudlet.setSizes(1024)
                    .setUtilizationModel(utilizationModel);
            list.add(cloudlet);
        }
        return list;
    }
}
```

Step 7: Run and Test the Simulation

Run the Program

1. Right-click on **BasicSimulation.java**
2. Select **Run As → Java Application**
3. Check the **Console** tab at the bottom of Eclipse

Expected Output

You should see:

- "Starting CloudSim Plus Simulation..." message
- A formatted table showing cloudlet execution results with columns:
  - Cloudlet ID
  - Status
  - Datacenter
  - VM ID

    - Time
    - Start Time
    - Finish Time
    - Length
- "Simulation finished!" message

Common Issues and Solutions

If you get NoClassDefFoundError again:

- Repeat Step 5 (Manual Dependency Purge)
- Clean the project: **Project** → **Clean...** → Select your project → **Clean**
- Rebuild and run again

If Maven build fails:

- Check your internet connection (Maven needs to download libraries)
- Try again - sometimes Maven Central repository is slow

Summary

You now have a fully working CloudSim Plus environment! The simulation creates:

- **2 Physical Hosts** with 8 processing elements each[10] [9]
- **2 Virtual Machines** running on the hosts
- **4 Cloudlets** (tasks) distributed across the VMs

The key difference in this procedure is **Step 5**, which uses the manual dependency purge method to ensure all CloudSim Plus libraries are properly downloaded and added to your project's classpath.[5] [6]

⁂

1. https://www.oracle.com/java/technologies/downloads/
2. https://www.youtube.com/watch?v=5fEmaUKTKUY
3. https://stackoverflow.com/questions/3017695/how-can-i-configure-encoding-in-maven
4. https://maven.apache.org/plugins/maven-resources-plugin/examples/encoding.html
5. https://www.cloudhadoop.com/maven-force-update
6. https://crunchify.com/mavenmvn-clean-install-update-project-and-project-clean-options-in-eclipse-ide-to-fix-any-dependency-issue/
7. https://stackoverflow.com/questions/10568275/noclassdeffounderror-on-maven-dependency/10568348
8. https://learn-it-university.com/troubleshooting-noclassdeffounderror-when-using-maven-dependencies/

9. https://maven.apache.org/xsd/maven-4.0.0.xsd
10. https://cloudsimplus.org/